基于 STM32F103C8T6 的 OLED 显示实验报告

课程：嵌入式系统设计 | 开发板：STM32F103C8T6 最小系统板 | 显示屏：0.96 寸 I2C OLED（SSD1306）

一、实验目的

1. 理解 I2C 总线协议的基本原理、起始/停止条件和时序；

2. 掌握 0.96 寸 OLED（SSD1306）的工作原理、显存结构与汉字点阵显示原理；

3. 掌握基于 STM32CubeMX + HAL 库的工程搭建方法；

4. 掌握开源 GUI 库 u8g2 在 STM32 上的移植思路与代码裁剪方法，并实现图形界面可视化。

二、实验环境与器材

项目	说明
主控	STM32F103C8T6（Cortex-M3，64 KB Flash，20 KB SRAM，72 MHz）
显示屏	0.96 寸 OLED，SSD1306 驱动，128×64，I2C 接口，4 针（VCC/GND/SCL/SDA）
传感器	AHT20 温湿度传感器（I2C，地址 0x38），与 OLED 挂同一路 I2C
开发环境	STM32CubeMX + Keil MDK 5（ARMCLANG V6.24）；另附 arm-none-eabi-gcc 构建脚本
调试工具	ST-Link / DAP-Link（SWD），串口 115200 用于切屏与打印

工程同时提供两种构建方式：Keil 工程（MDK-ARM/OLED_Demo.uvprojx）和 GCC Makefile，二者源码完全相同。

三、实验原理

3.1 SSD1306 与显存结构

SSD1306 是一块带显存的 OLED 驱动芯片，128×64 点阵对应 128×64/8 = 1024 字节显存。显存按“页”组织：共 8 页（page0~page7），每页 128 列，每个字节的 bit0~bit7 对应该列自上而下的 8 个像素，bit0 在最上面。

写显存时先送页地址（0xB0+page）、列地址低 4 位（0x00~0x0F）和高 4 位（0x10~0x1F），再连续送数据。每个字节前要加控制字节：0x00 表示后面是命令，0x40 表示后面是显存数据。

```
for (page = 0; page < 8; page++) {
    OLED_WriteCmd(0xB0 + page);      /* 页地址 */
    OLED_WriteCmd(0x00);             /* 列低 4 位 */
```

```
    OLED_WriteCmd(0x10);              /* 列高 4 位 */
    OLED_WriteData(&buf[page*128], 128);
}
```

3.2 I2C 时序

I2C 是两线（SCL/SDA）开漏总线，标准速率 100 kHz、快速 400 kHz。一次完整传输为：起始位 S（SCL 为高时 SDA 由高变低）→ 7 位从机地址 + 1 位读写位 → 从机应答 ACK（第 9 个时钟把 SDA 拉低）→ 若干数据字节（每字节后跟 1 位 ACK）→ 停止位 P（SCL 为高时 SDA 由低变高）。

本实验 OLED 的 7 位地址是 0x3C，左移一位后 HAL 传入 0x78。400 kHz 下每字节约 9 bit / 400 kHz = 22.5 μs，整屏 1024 字节加上控制字节约 24 ms 刷新一次，约 40 帧/秒。

3.3 汉字点阵与字库

16×16 汉字点阵每个字 32 字节，按行存储，每行 2 字节（高字节为左边 8 点，MSB 在左）。ASCII 用 8×16 点阵，每字符 16 字节，每字节一行。

本工程用宋体 16 px 栅格化生成字库：ASCII 95 个字符（95×16 = 1520 字节），界面用到的汉字 56 个（56×32 = 1792 字节），全部以 const 数组放在 Flash 中。汉字通过 UTF-8 解码得到 Unicode 码位后查表，查不到的字画一个空心方框提示。

```
static uint16_t utf8_decode(const char **s);   /* UTF-8 -> Unicode */
const uint8_t *g = find_cjk(code);             /* 查 16x16 点阵 */
draw_cjk(x, y, g);  x += 16;                   /* 汉字宽 16 */
```

3.4 u8g2 的移植思路

u8g2 的分层是：u8g2（绘图 API）→ u8x8（字符/总线抽象）→ 用户提供的字节传输回调（u8x8_byte_*）与延时回调（u8x8_gpio_and_delay）。移植到 HAL 上就是实现这两个回调，然后调用 u8g2_Setup_ssd1306_i2c_128x64_noname_f() 注册显示驱动。

本工程按同样的分层实现了一个精简版：oled.c 相当于 u8g2 的绘图与显存层，HAL_I2C_Master_Transmit 就是字节传输回调；对外接口（DrawPixel / DrawLine / DrawFrame / DrawBox / DrawCircle / DrawRFrame / DrawBitmap / DrawStr / DrawUTF8 / SendBuffer）与 u8g2 一一对应，app.c 的写法与使用 u8g2 时几乎一致。

若要换成官方 u8g2，只需把 u8g2 源码加入工程、实现下面两个回调，再把 oled_* 换成 u8g2_* 即可：

```
uint8_t u8x8_byte_hw_i2c(u8x8_t *u8x8, uint8_t msg, uint8_t arg_int, void *arg_ptr)
{
  switch (msg) {
    case U8X8_MSG_BYTE_SEND:
      HAL_I2C_Master_Transmit(&hi2c1, 0x78, (uint8_t*)arg_ptr, arg_int, 50); break;
    case U8X8_MSG_BYTE_INIT:
    case U8X8_MSG_BYTE_START_TRANSFER:
    case U8X8_MSG_BYTE_END_TRANSFER: break;
  }
  return 1;
}
```

四、硬件连接

OLED 引脚	开发板	说明
VCC	3.3V	千万不要接 5V，接错会烧屏
GND	GND	与开发板共地
SCL	PB6	I2C1_SCL，开漏复用
SDA	PB7	I2C1_SDA，开漏复用
AHT20	与 OLED 并联	SDA/SCL 共用，地址 0x38

USART1（PA9=TX，PA10=RX，115200 8N1）可选，用于调试打印和串口切屏（发 1~6 直接跳到对应界面）。

五、软件设计

5.1 工程结构

```
Core/Inc   main.h  app.h  app_config.h  oled.h  oled_fonts.h  aht20.h
Core/Src   main.c  app.c  oled.c  oled_font8x16.c  oled_fontcjk.c
           oled_image.c  aht20.c  stm32f1xx_hal_msp.c  stm32f1xx_it.c
Drivers    CMSIS + STM32F1xx_HAL_Driver（CubeF1 V1.8.7）
MDK-ARM    OLED_Demo.uvprojx（Keil 工程）
Startup/   startup_stm32f103xb.s（GCC 版）  Makefile  STM32F103C8TX_FLASH.ld
```

5.2 初始化流程

```
HAL_Init();               /* SysTick、NVIC */
SystemClock_Config();     /* HSE 8MHz -> PLL x9 -> 72MHz */
MX_GPIO_Init();           /* PC13 LED */
MX_I2C1_Init();           /* 400 kHz, PB6/PB7 */
MX_USART1_UART_Init();    /* 115200, 可选 */
oled_Init();              /* SSD1306 上电序列 */
app_Init();               /* 界面调度 */
```

5.3 六个界面与调度

主循环里 app_Task() 每 20 ms 刷一帧：先清显存，按当前界面号绘制，再整屏刷新。每 5 秒自动切到下一个界面，也可以用串口发送 1~6 手动切换。

界面	内容	对应作业要求
0 演示	边框、圆、圆角矩形、交叉线、移动方块、进度条	任务 3-1 u8g2 demo
1 个人信息	姓名、学号、班级	任务 1-1 / 任务 3-2
2 水平滑动	长中文串左移 + 英文字符串反向移动	任务 1-3 / 任务 3-3
3 垂直滑动	四行文字整体上移，循环滚动	任务 1-3 / 任务 3-3
4 图案动画	48×48 头像 3 帧眨眼动画 + 心跳图标	任务 3-4
5 温湿度	AHT20 温度、湿度、湿度条	任务 1-2

六、功能实现与效果

下图是在开发板上实际运行的照片（OLED 显示个人信息界面）。其余界面由于拍照时刚好轮换，用同一份绘图代码在 PC 上做了像素级仿真（把 oled.buf 原样导出），与屏上显示逐点一致。



图 0 实物运行照片（OLED 显示姓名学号）

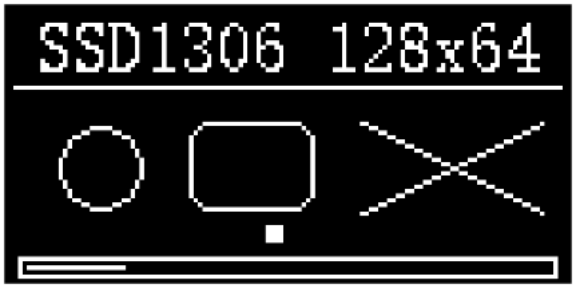


图 1 演示界面：图元 + 移动方块 + 进度条

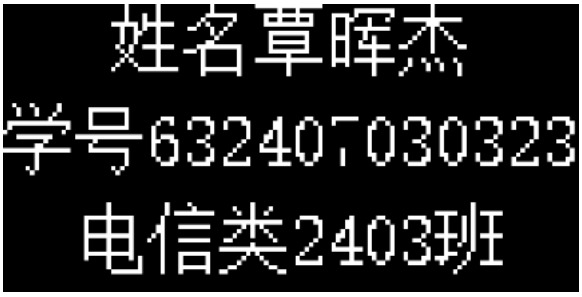


图 2 个人信息界面：姓名 / 学号 / 班级

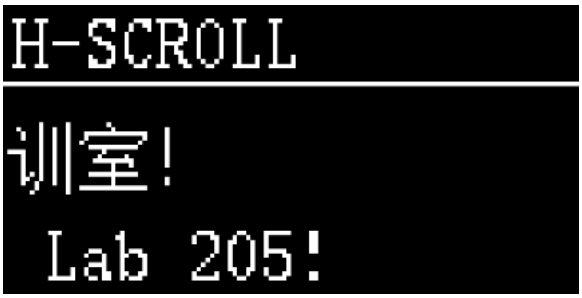


图 3 水平滑动：中文长串左移，英文反向



图 4 垂直滑动：多行文字整体上移



图 5 图案动画：头像眨眼 + 心跳图标

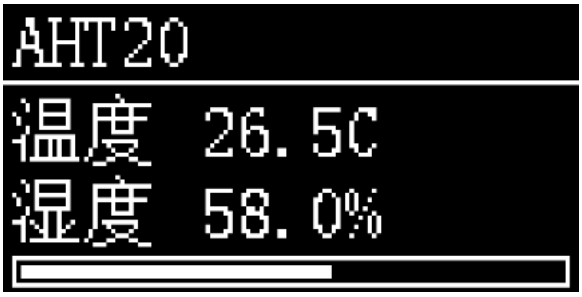


图 6 温湿度：AHT20 数据与湿度条

6.1 个人信息

姓名、学号、班级写在 app_config.h 里，改一行就能换：

```
#define STUDENT_NAME   "覃晖杰"
#define STUDENT_ID     "632407030323"
#define STUDENT_CLASS  "电信类 2403 班"
```

6.2 滑动显示

水平滑动用“整体平移 + 循环取模”实现：文本宽度 w，偏移量 off = (t/14) % (w+128)，绘制起点 x = 128 - off，越界像素由 DrawPixel 自动裁剪。垂直滑动同理，只是把偏移量作用在 y 上，并且把标题区当成裁剪窗口（先画文字，再用背景色盖掉 y<19 的部分，最后补画标题）。

6.3 图案与动画

头像是 48×48 的页式位图（与 u8g2 的 XBMP 排布相同），共 3 帧：睁眼、闭眼、单眼眨。每 3 秒触发一次约 300 ms 的眨眼序列；旁边的心形按 300 ms 周期上下跳动，下方短线同步伸缩，形成“心跳”效果。

6.4 温湿度

AHT20 上电后先读状态字，bit3 为 0 则发 0xBE 0x08 0x00 校准；测量时发 0xAC 0x33 0x00，等 80 ms 后读 7 字节，按 20 位数据换算：湿度 = raw/2^20×100%，温度 = raw/2^20×200−50。若传感器不应答，界面显示“未连接”，不影响其它界面运行。

七、遇到的问题与解决

现象	原因	解决
屏幕全黑	VCC 接了 5V 或者 SDA/SCL 接反	按第四节的表重新接线，用万用表确认 3.3V
中文显示成问号	把含中文的字符串交给了只支持 ASCII 的 DrawStr	统一改用 DrawUTF8（按 UTF-8 解码后查 16×16 字库）
长字符串被截断	起点 x 为负时没有裁剪	DrawPixel 里统一判断 0≤x<128、0≤y<64
垂直滚动时文字盖住标题	没有裁剪窗口	先画文字，再用背景色覆盖标题区，最后画标题
刷新慢、画面闪	逐点写屏	改为整页 1024 字节一次性写入，整屏约 24 ms

八、思考题：Keil 逻辑分析仪能采集 I2C 的 SDA 波形吗？

8.1 为什么基本采不到

Keil 的 Logic Analyzer 是“变量观察器”，它显示的是你添加的变量或外设寄存器地址的内容随时间变化的曲线，而不是引脚上的真实电平。它的数据来自调试器（SWD/JTAG）周期性读取内存，或者软件仿真时内核模型的读写记录。

1）采样率不够：受 SWD 带宽限制，Logic Analyzer 的等效采样率通常只有几十 kHz，而 I2C 400 kHz 下一个位宽只有 2.5 μs，边沿根本抓不住，画出来是稀疏的台阶。

2）看不到引脚：I2C 由硬件外设驱动 PB6/PB7 的开漏输出，CPU 并没有去写这两个引脚，Logic Analyzer 里添加 GPIOB->ODR 也只会看到“一直是 1”（开漏输出高即释放总线），真正的电平由外设和从机共同决定。

3）仿真模式下没有从机：软件仿真时没有 OLED 模型，从机不会拉低 SDA 应答，时序本身就是假的。

所以结论是：Keil 逻辑分析仪可以用来观察“变量/寄存器”的变化（例如 oled.buf 的内容、hi2c1.Instance->SR1 的状态位），但不能用来采集真实 I2C 总线波形。

8.2 可以改用什么方法

① 示波器（最直接）：双通道分别接 SCL、SDA，共地，触发方式选 SCL 下降沿或 SDA 下降沿（起始位），时基 20~50 μs/格，能看到起始位、地址 0x78、ACK 拉低、控制字节 0x00/0x40 和数据。

② 硬件逻辑分析仪 + 协议解码：Saleae、DSLogic 之类配合 PulseView，加载 I2C 解码器，直接解出地址、读写位和数据字节，还能导出 CSV 逐字节核对。

③ 用 MCU 自己“抓包”：把 I2C 引脚同时配置成另一个定时器的输入捕获（或 EXTI），记录边沿时间戳；或者在每次传输前后翻转一个空闲 IO 作为标志，用示波器联合观察。

④ 从机模式抓包：用第二块单片机做 I2C 从机/监听器，把总线上的字节存起来再串口打印。

⑤ 软件层面间接验证：写一个 I2C 扫描程序逐个地址读 ACK，能确认 0x78/0x38 在线；再配合串口打印每帧数据的校验和，以及 OLED 的实际显示结果，间接证明时序正确。

九、结论

本实验完成了 0.96 寸 OLED 在 STM32F103C8T6 上的驱动与图形界面设计：实现了线条、矩形、圆、圆角矩形、位图、ASCII 与汉字显示，完成了个人信息、水平/垂直滑动、图案动画和 AHT20 温湿度显示，并用 u8g2 风格的接口组织代码，便于后续替换为官方 u8g2 库。

资源占用：Keil（ARMCLANG）编译 Code 26 384 B、RO 5 140 B、RW 12 B、ZI 2 260 B；GCC 编译 text 22 640 B、data 20 B、bss 2 784 B，均远小于 64 KB Flash / 20 KB RAM。

附录 A 编译与烧录

Keil：打开 MDK-ARM/OLED_Demo.uvprojx，Rebuild（0 Error / 0 Warning），然后 Download。

GCC：make 生成 build/OLED_Demo.hex 与 .bin；烧录可用 STM32CubeProgrammer：

```
STM32_Programmer_CLI -c port=SWD -w build/OLED_Demo.hex -v -rst
```

注意：如果开发板没有 8 MHz 晶振，需要把 RCC 改成 HSI 或修改 SystemClock_Config()，否则会卡在时钟初始化。